



ZED X Drivers Development Guide

Revision v1.0 - 04/2026

Table of contents

Introduction	2
Prerequisites and Environmental Setup	3
Access and Repository Setup	3
Toolchain and Kernel Source Management	3
A Guided Tour of the Repository	5
Driver Source Code Location	5
Device Tree Location and Management	5
Daemon	5
repack	6
How to build	7
Usage and Options	7
Example Usage	7
How to package	8
Usage and Options	8
Example Usage	8
Custom Kernel Sources Integration	9
Device tree customization	10
Component description	10
Deserializers	10
MAX9296	10
MAX96712	11
Dummys	12
Serializers	12
MAX9295	12
Sensors	14
ZED X / ZED X One GS	14
ZED X HDR / ZED X One HDR	17
ZED X One 4K	18
IMUs	19
Architecture description	20
Tegra capture VI	20
I2C Control Channels	23
Deserializer MIPI outputs	24

Introduction

This guide serves as a comprehensive, step-by-step resource for our partners to successfully develop custom drivers for integrating our camera hardware with their Jetson based ECU platforms.

The successful integration of our high-performance cameras relies on robust and optimized software drivers tailored to the specific architecture and operating system of your target ECU. This document breaks down the complex process into manageable phases, ensuring a clear pathway from initial setup and understanding of hardware interfaces to final driver implementation and validation. Successful driver implementation is critically dependent on adhering to the specific hardware interface constraints and integration rules detailed in the **Reference Design Guide**

Failure to strictly follow the guidelines presented in the reference design will directly lead to sub-optimal performance, including reduced frame rates and/or resolution and number of connectable cameras. In certain cases, deviations from these established hardware rules may result in complete incompatibility between the camera module and the host ECU platform. We strongly advise partners to treat the reference design guide as a mandatory specification for all hardware-level integration efforts.

To have your newly developed driver approved and potentially shared on our website, please complete the necessary **certification** process. Boards with approved drivers will be given priority when advising customers who have specific board requirements.

Prerequisites and Environmental Setup

The foundational step for driver development is setting up the correct software environment. This step ensures you have access to the necessary source code, tools, and libraries aligned with your target hardware and operating system.

Access and Repository Setup

1. **Repository Access:** Access to our private GitHub repository is required. This access is provided on demand by email.
2. **Cloning the Repository:** Once access is granted, clone the repository to your development host machine:

None

```
git clone git@github.com:stereolabs/zedx-driver.git
```

If git-lfs is crashing, you can bypass it by using :

None

```
GIT_LFS_SKIP_SMUDGE=1 git clone git@github.com:stereolabs/zedx-driver.git
```

3. **Initial Environment Setup:** Execute the preparation script to configure essential environment variables:

None

```
./prepare.sh
```

Toolchain and Kernel Source Management

1. **Manual Source Download (If LFS Fails):** In instances where `git-lfs` fails to download the necessary kernel sources, you can perform the download manually:

None

```
./download_nvidia_src.sh <L4T_Version>
```

(Replace <L4T_Version> by the L4T release you want to build for (ex : R36.4 or R35.3)

Supported L4T_Version: R39.2 / R38.4 / R38.2 / R36.5 / R36.4 / R36.3 / R35.4 / R35.3

2. **Installing Sources:** Whether it is after a manual download or after a successful `git-lfs` pull, run the `switch_release.sh` script to install the source files and properly configure your environment:

None

```
./switch_release.sh <L4T_Version>
```

(Replace <L4T_Version> by the L4T release you want to build for (ex : R36.4 or R35.3)

This command will set up some variables and apply patches to set up all the configuration files properly to build your driver properly.

A Guided Tour of the Repository

Driver Source Code Location

The core driver source code resides within the `src/kernel/stereolabs` directory.

- **Location:** `src/kernel/stereolabs`
- **Content:** Contains all proprietary source files for the GMSL camera drivers.
- **Maintenance:** This folder must remain intact and unmodified.

If a new feature is required or if a bug is discovered, partners must contact the development team directly by creating a new issue using the "Issue" tab within the private repository. **Do not modify the files within the `src/kernel/stereolabs` folder directly.**

Device Tree Location and Management

The device tree source files, which are crucial for defining the camera hardware's configuration and interconnectivity on the ECU, are located in the `src/hardware/stereolabs` directory.

- **Location:** `src/hardware/stereolabs`
- **Content:** This directory contains the following key components:
 - **Makefile:** Used for compiling the device tree overlays.
 - **Source Files:** Two primary source files: one defining the I2C configuration of the device, and the other managing the streaming pipeline.
 - **Utils Folder:** This folder contains files detailing all camera working modes. **It must remain unmodified.**
 - **Template Folder:** Includes templates of device tree files that partners can use as a starting point for their specific integrations.

Daemon

The Daemon source code is stored in the `Daemon/` directory. It is used for driver loading at boot and restarting in case of a camera crash. This tool is deeply intertwined with the SDK to ensure stream stability and data retrieval.

For your driver to be certified, Daemon has to be included in your package.

- **Location:** `Daemon/`
- **Content:** This directory contains the following key components:
 - **zed_x_daemon:** Main service used to pilot sub-service in a blocking process to ensure data is not used before the whole restart of the driver is achieved.

- **driver_zed_loader**: service in charge of reloading the video stream related drivers (deser/ser/sensor)
- **Imu-daemon**: service in charge of the IMU related drivers.
- **Maintenance**: This folder must remain intact and unmodified.

If a new feature is required or if a bug is discovered, partners must contact the development team directly by creating a new issue using the "Issue" tab within the private repository. **Do not modify the files within the **Daemon/** folder directly.**

repack

To create a new Debian package, we use the **repack/** folder, where you'll find all the processes run during driver installation. Apart from copying modules and device tree to their right place in the board rootfs, the processes executed set some critical environment variables and kernel patches required for optimal performance.

- **Location**: **repack/**
- **Content**: This directory contains the following key components:
 - **preinst**: paste all StereoLabs-related files in the board rootfs.
 - **Postinst**: apply environment variables and patch configuration files
 - **postrm**: cleanup of kernel patches
- **Maintenance**: This folder must remain intact and unmodified.

If a new feature is required, or if a bug is discovered, partners must contact the development team directly by creating a new issue using the "Issue" tab within the private repository. **Do not modify the files within the **repack/** folder directly.**

How to build

The `./build.sh` script is the primary utility for compiling the custom driver components. It manages the build process for the kernel image, kernel modules, and the Device Tree Blob, storing all resulting artifacts in the `output/` directory.

Usage and Options

Option	Type	Description
<code>-h</code>		Displays the help message (this guide).
<code>-r</code>	Boolean	Builds the driver components to be compatible with a Real-Time (RT) Linux kernel.
<code>-ov</code>	Boolean	Builds a DTBO file (Device Tree Blob Overlay). Compiles a DTB file if not specified.
<code>-dt</code>	Path	Specifies the Stereolabs camera master device tree to be used for the build.
<code>-pdt</code>	Path	Specifies the Parent Device Tree file to be used, required if you are building a full <code>.dtb</code> instead of an overlay.

In cas of a .dtb build, the script will automatically modify the parent devicetree file to take yours into account.

!! The devicetree has to be stored in `src/hardware/stereolabs/` to be taken into account

Example Usage

1. Standard Driver and Overlay Build:

```
None
./build.sh -ov /path/to/sl_devicetree.dts
```

(This is the most common use case after performing device tree customization.)

2. Building for Real-Time Kernel and DTB build:

```
None
./build.sh -r /path/to/sl_devicetree.dts -pdt /path/to/your/parent.dts
```


How to package

The final step is to package the compiled artifacts into a deployable Debian package (`.deb`). This package includes the kernel modules, the Device Tree Blob/Overlay, the Daemon services, and the necessary scripts for installation and configuration (`repack/` contents).

The `./package.sh` script automates the entire process, including running `./build.sh` and assembling the final Debian package using the contents of the `repack/` directory.

Usage and Options

Option	Type	Description
<code>-h</code>		Displays this help message.
<code>-r</code>	Boolean	Builds the driver components to be compatible with a Real-Time (RT) Linux kernel.
<code>-ov</code>	Boolean	Builds a DTBO file (Device Tree Blob Overlay). Compiles a DTB file if not specified.
<code>-dt</code>	Path	Specifies the Stereolabs camera master device tree to be used for the build.
<code>-pdt</code>	Path	Specifies the Parent Device Tree file to be used, required if building a full <code>.dtb</code> instead of an overlay.
<code>-n</code>	String	Specifies the name for the final Debian package. stereolabs-<Package_name>_<Versioning>.deb

Example Usage

1. Standard Package Build (Overlay):

None

```
./package.sh -n my-custom-camera-driver -ov /path/to/camera_overlay.dts
```

(This creates the package `stereolabs-my-custom-camera-driver_<version>.deb` containing the overlay and standard modules.)

2. Package Build for Real-Time Kernel (DTB):

None

```
./package.sh -n rt-camera-driver -r -dt /path/to/sl_devicetree.dts -pdt  
/path/to/devicetree_parent.dts
```

(This creates an RT-compatible package and builds a full DTB.)

Upon successful execution, the final `.deb` package will be located in the `zedx-driver/` directory, ready for transfer and installation on the target ECU.

Custom Kernel Sources Integration

When utilizing custom modifications within the NVIDIA kernel source code to accommodate specific hardware designs, the resulting kernel modules may exhibit versioning incompatibilities.

This typically manifests during the module loading process as a symbol version mismatch error:

None

```
sl_zedx : disagrees about version of symbol...
```

To resolve this discrepancy, the entire custom kernel source tree must be packaged into an archive named `kernel_R<L4T_Version>.zip` and placed within the `nvidia_kernel/src_archives` directory.

Subsequently, execute the `./switch_release.sh` script for the intended L4T version to reconfigure the build environment. If a `build_R<L4T_Version>` directory is already present, it must be removed to ensure a clean integration process.

Apart from the kernel source code, this principle also applies to custom devicetrees.

Device tree customization

Component description

Deserializers

MAX9296

None

```
max9296_a@48 {
    compatible = "stereolabs, sl_max9296";
    reg = <0x48>;
    channel = "a";
    sync_mode = "master";
    pwn-gpio = < &gpio CAM1_PWDN GPIO_ACTIVE_HIGH >;
    camera-serializers = < &zedx_ser0 &zedxone_gs_ser0 &zedxone_uhd_ser0 >;
};
```

Property	Type	Description
<code>compatible</code>	String	Must be set to <code>"stereolabs, sl_max9296"</code> to bind this device tree node to the proprietary Stereolabs deserializer driver. Do not modify.
<code>reg</code>	Integer	The I2C address of the MAX9296 chip. This value is hardware-fixed and must match the physical configuration.
<code>channel</code>	String	In case of multiple deserializer architecture, specify the channel on the deserializer being configured. Common values are <code>"a"</code> , <code>"b"</code> , <code>"c"</code> , or <code>"d"</code> .
<code>sync_mode</code>	String	Defines the synchronization role of this deserializer. Set to <code>"master"</code> for the device that controls the frame synchronization timing for the connected serializers. The other deserializer has to be set as <code>"slave"</code> .
<code>pwn-gpio</code>	Phandle/GPIO	Specifies the GPIO pin used for controlling the Power Down (PWDN) state of the chip. This must reference a valid GPIO line on your ECU.
<code>camera-serializers</code>	Phandle list	A list of references (phandles) to the connected Serializer device tree nodes (<code>&zedx_ser0</code> , etc.). This establishes the camera-to-deserializer topology.

MAX96712

None

```
max96712_a@29 {
    compatible = "stereolabs, sl_max96712";
```

```

reg = <0x29>; /* deserializer i2c_address */
channel = "a"; /* deserializer channel: one per deser */

camera-serializers = <
    /* GMSL 0 */ &zedx_ser0 &zedxone_gs_ser0 &zedxone_uhd_ser0
    /* GMSL 1 */ &zedx_ser1 &zedxone_gs_ser1 &dummy_zedxone_uhd_ser
    /* GMSL 2 */ &zedx_ser2 &zedxone_gs_ser2 &zedxone_uhd_ser2
    /* GMSL 3 */ &zedx_ser3 &zedxone_gs_ser3 &dummy_zedxone_uhd_ser >;
};

```

Property	Type	Description
<code>compatible</code>	String	Must be set to <code>"stereolabs, sl_max9296"</code> to bind this device tree node to the proprietary Stereolabs deserializer driver. Do not modify.
<code>reg</code>	Integer	The I2C address of the MAX9296 chip. This value is hardware-fixed and must match the physical configuration.
<code>channel</code>	String	In case of multiple deserializer architecture, specify the channel on the deserializer being configured. Common values are <code>"a"</code> , <code>"b"</code> , <code>"c"</code> , or <code>"d"</code> .
<code>camera-serializers</code>	Phandle list	A list of references (phandles) to the connected Serializer device tree nodes (<code>&zedx_ser0</code> , etc.). This establishes the camera-to-deserializer topology.

Dummys

Due to hardware limitations, you might need to declare dummies to limit the maximum number of cameras per deserializer. In the example used just before, we see that only 2 ZED X One UHD can be connected to a MAX96712. In order to limit the maximum number of cameras connectable, 2 dummies have been added to the list of serializers.

```

None
zedxone_uhd:zedxone_uhd@1a {};
dummy_zedxone_uhd_ser: zedxone_uhd_ser@44 {
    camera_model = "zedone4k";
    zedx-id = "-1";
    camera-sensors = <&zedxone_uhd>;

```

```
};
```

The `zedx-id = "-1"` is the parameter that sets this entry as a dummy. The `camera_model` has to be set as well, meaning if you need multiple camera types to be dummies, you'll have to create one entry by `camera_model`.

Serializers

MAX9295

Stereo camera :

```
None
zedx_ser0: zedx_ser_0@62 {
    compatible = "stereolabs,sl_max9295";
    reg = <0x62>;

    zedx-id = "0";
    channel = "a";
    mode = "master";

    camera_model = "zedx";
    camera-sensors = <&zedx_left0 &zedx_right0>;
    imu = <&bmi088_a &bmi088_b>;
};
```

Monocular camera :

```
None
zedxone_gs_ser0: zedxone_gs_ser_0@66 {
    compatible = "stereolabs,sl_max9295";
    reg = <0x66>;

    zedx-id = "4";
    channel = "a";
    mode = "master";

    camera_model = "zedonegs";
    camera-sensors = <&zedxone_gs0>;
    imu = <&bmi088_a &bmi088_b>;
};
```

Property	Type	Description
compatible	String	Must be set to " stereolabs,sl_max9295 " to bind this device tree node to the proprietary Stereolabs serializer driver. Do not modify.
reg	Integer	The I2C address of the MAX9295 chip. This value is hardware-fixed and must match the physical camera configuration.
zedx-id	String	A unique identifier for the camera module connected to this serializer (e.g., " 0 " for the first camera). Must be unique per serializer.
channel	String	The channel on the serializer being configured. Common values are " a ", " b ", " c ", or " d ". Value have to be the same as the linked deserializer
mode	String	Defines the synchronization role of the serializer. Always set to "master"
camera_model	String	Specifies the model of the camera connected (e.g., " zedx ", " zedxone_gs ", " zedone4k ", " zedxhdr ", " zedonehdr "). This is used to load the correct camera parameters and operating modes.
camera-sensors	Phandle list	A list of references (phandles) to the connected camera sensor device tree nodes (e.g., &zedx_left0 , &zedx_right0). For a monocular camera, list the only corresponding sensor (e.g., zedxone_gs0). Required.
imu	Phandle list	A list of all references (phandles) to the connected IMU device tree nodes linked to the i2c_bus (e.g., &bmi088_a , &bmi088_b). IMU allocation will be done dynamically in the device tree, hence the need to list all of them in this vector.

Sensors

ZED X / ZED X One GS

Material description :

```
None
zedx_left0: zedx_left_0@28 {
```

```

compatible = "stereolabs,zedx";
reg = <0x28>;

ZED_ORIN_CLKS

zedx-id = "0";
channel = "a";

has-eprom;
eprom-addr=<0x54>;
};

```

Property	Type	Description
<code>compatible</code>	String	Must be set to "stereolabs,zedx" to bind this device tree node to the proprietary Stereolabs sensor driver. Do not modify.
<code>reg</code>	Integer	The I2C address of the camera sensor chip (e.g., <code>0x28</code>). This value is hardware-fixed and must match the physical configuration.
<code>ZED_ORIN_CLKS</code>	Macro	A placeholder macro used to define the required clock configuration specifically for the NVIDIA Jetson Orin platform. Set entry depending on the Jetson generation (e.g., <code>ZED_ORIN_CLKS</code> , <code>ZED_XAVIER_CLKS</code> , <code>ZED_THOR_CLKS</code>)
<code>zedx-id</code>	String	A unique identifier for the camera module this sensor belongs to (e.g., <code>"0"</code>).
<code>channel</code>	String	Specifies the channel on the serializer/deserializer this sensor is connected to (e.g., <code>"a"</code>).
<code>has-eprom</code>	Boolean	Indicates that the camera module includes an EEPROM for storing calibration data.
<code>eprom-addr</code>	Integer	The I2C address of the EEPROM chip (e.g., <code>0x54</code>).

Stream pipping :

```

None
zedx_left0: zedx_left_0@28 {

```

```

devnode = "video0";

/* ZED SDK related variables */
sync_sensor = "ZED_CAM0";
sync_sensor_index = <1>;

AR0234_MODES("0", "serial_a", "0")

ports {
    #address-cells = <1>;
    #size-cells = <0>;
    port@0 {
        reg = <0>;
        zedx_ar0234_out0: endpoint {
            vc-id = <0>;
            port-index = <0>;
            opt-csi-port = "c";
            bus-width = <NUM_LANES>;
            i2c-cc = <0>;
            remote-endpoint = <&zedx_csi_in0>;
        };
    };
};
};

```

This entry defines the data stream path, or "pipping," from the camera sensor to the host system's CSI (Camera Serial Interface) receiver port. This configuration is essential for image data transfer.

Property	Type	Description
devnode	String	Specifies the Linux device node that will represent this video stream (e.g., "video0"). Set a different devnode for each sensor
sync_sensor	String	A label used by the ZED SDK to define the synchronization group for this sensor (e.g., "ZED_CAM0").
sync_sensor_index	Integer	A numerical index used by the ZED SDK for multi-sensor synchronization (e.g., <1>).

Property	Type	Description
AR0234_MODES	Macro	A placeholder macro that defines the sensor's supported operating modes (resolution, frame rate, etc.). The parameters specify the VC-ID, the Jetson serial input, and the lane_polarity. Lane_polarity = "0" except on Jetson Orin NX and Nano on serial_a and serial_b, where Lane_polarity = "6". Use the entry corresponding to the right camera type - ZED X & ZED X One GS: AR0234_MODES - ZED X HDR & ZED X One HDR: ISX031_MODES - ZED X One 4k: IMX678_MODES

Porting configuration :

Property	Type	Description
vc-id	Integer	Virtual Channel ID used in the CSI-2 protocol.
port-index	Integer	The index of the CSI port on the Jetson side.
opt-csi-port	String	Specifies the physical CSI port out of the deserializer (e.g., "c", "d", "e", "f").
bus-width	Macro	Defines the number of data lanes used in the CSI connection (NUM_LANES).
i2c-cc	Integer	The I2C control channel index. (e.g. 0 or 1)
remote-endpoint	Phandle	A reference (<&zedx_csi_in0>) that links this sensor output to the corresponding NVCSI channel.

ZED X HDR / ZED X One HDR

Material description

```

None
zedx_hdr_right0: zedx_hdr_right_0@2e {
    compatible = "stereolabs,zedxhdr";
    reg = <0x2e>;

    ZED_ORIN_CLKS

    zedx-id = "10";
    channel = "a";

```

```

    has-eprom;
    eeprom-addr = <0x52>;
};

```

Stream pipping

None

```

zedx_hdr_left0: zedx_hdr_left_0@3e {

    devnode = "video7";

    sync_sensor = "ZED_CAM4";
    sync_sensor_index = <1>;

    ISX031_MODES("0", "serial_a", "0")

    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {
            reg = <0>;
            zedxhdr_isx031_out0: endpoint {
                vc-id = <0>;
                port-index = <0>;
                opt-csi-port = "c";
                bus-width = <NUM_LANES>;
                i2c-cc = <0>;
                remote-endpoint = <&zedxhdr_csi_in0>;
            };
        };
    };
};

```

ZED X One 4K

ZED X One 4K camera is designed with a fixed I2C address, specifically set to **0x1a**. This hard-coded address is a fundamental hardware constraint of the device.

This fixed I2C address dictates that only one ZED X One 4K camera can be present on any single I2C bus segment. The I2C protocol relies on unique addresses for proper device communication and arbitration. If multiple devices with the same address are connected to the same bus, the host controller cannot reliably distinguish between them, leading to communication conflicts and system failure.

To accommodate the necessary use of multiple ZED X One 4K cameras within a larger system, system architects must ensure that each camera is placed on a logically or physically separate I2C bus (TCAs, multiple I2C channels). To ensure that multiple cameras can't be connected to a single I2C bus, we added dummies to the deserializer camera-serializers list.

Material description

```
None
zedxone_uhd0:zedxone_uhd_0@1a {
    compatible = "stereolabs,zedxone_uhd";
    reg = <0x1a>;

    ZED_ORIN_CLKS

    zedx-id = "8";
    channel = "a"; /* deserializer channel */

    has-eeeprom;
    eeeprom-addr = <0x54>;
};
```

Stream pipping

```
None
zedxone_uhd0:zedxone_uhd_0@1a {

    devnode = "video6";

    sync_sensor = "ZEDXONE_CAM4";
    sync_sensor_index = <1>;

    IMX678_MODES("0", "serial_a", "0")

    ports {
        #address-cells = <1>;
    }
};
```

```

        #size-cells = <0>;
        port@0 {
            reg = <0>;
            imx678_imx678_out0: endpoint {
                vc-id = <0>;
                port-index = <0>;
                opt-csi-port = "c";
                bus-width = <NUM_LANES>;
                i2c-cc = <0>;
                remote-endpoint = <&imx678_csi_in0>;
            };
        };
    };
};

```

IMUs

```

None
bmi088_a: bmi088_a@3b {
    status = "okay";
    compatible = "bmi,bmi088";
    reg = <0x3b>; /* <-- Must be gyroscope I2C address */
    accel_i2c_addr = <0x3c>; /* Must be specified */
    accel_matrix    = [01 00 00 00 01 00 00 00 01];
    gyro_matrix     = [01 00 00 00 01 00 00 00 01];
};

```

IMU Parameter Description

The IMU device tree entry defines the fundamental properties of the connected Inertial Measurement Unit (IMU) sensor.

Property	Type	Description
compatible	String	Must be set to the specific IMU model (e.g., "bmi,bmi088") to bind this device tree node to the appropriate kernel driver. Do not modify.

Property	Type	Description
reg	Integer	The I2C address of the IMU's gyroscope component (e.g., <code><0x3b></code>). This is the primary address for the IMU node.
accel_i2c_addr	Integer	The I2C address of the IMU's accelerometer component (e.g., <code><0x3c></code>). This must be explicitly specified if the accelerometer and gyroscope have different I2C addresses.
Accel_matrix gyro_matrix	Array of bytes	Do not modify.

Architecture description

Tegra capture VI

Establishing a functional video stream requires meticulous configuration across three interconnected kernel entities: the **Tegra Capture VI (Video Input)**, the **NVIDIA CSI (Camera Serial Interface)** block (nvcsi), and the **actual sensor porting** configuration. The video stream will not operate until all three components are correctly and harmoniously configured. The Video Stream Pipelining Logic

The data flow, or "remote endpoint pointing logic," for the video pipeline is directional and follows this essential sequence:

tegra-capture-vi <-- (links to) --> **nvcsi** <-- (links to) --> **sensor port**

This logic dictates how the components are registered and configured within the device tree or similar kernel configuration mechanisms. The **tegra-capture-vi** acts as the video sink, receiving data from the **nvcsi** block, which in turn receives the raw data from the

configured **sensor port**.

For the successful configuration and activation of the video piping, all three entities **must share the exact same configuration parameters** for the data link. This is the foundation of ensuring synchronization and proper data interpretation across the pipeline. The critical parameters that must be synchronized across all three components are:

1. **VC-ID (Virtual Channel ID):** This identifier is crucial for multi-stream scenarios where multiple camera sensors share the same physical MIPI CSI-2 bus. Each virtual channel acts as an independent data path. All three components (**tegra-capture-vi**, **nvcsi**, and **sensor port**) must be configured with the same VC-ID to ensure the VI block correctly pulls data intended for that specific sensor stream.
2. **Port Index:** This parameter defines which physical or logical port on the Tegra SoC the sensor is connected to. It is used by the **tegra-capture-vi** and **nvcsi** blocks to identify the hardware resources involved in the connection. This index must match the port index defined in the sensor's configuration.
3. **Bus Width (Lane Configuration):** This refers to the number of physical MIPI CSI-2 data lanes utilized for the stream (e.g., 2-lane or 4-lane configuration). A mismatch in the number of active lanes between the sensor's output (as defined in its porting layer) and the **nvcsi** and **tegra-capture-vi** configurations will result in corrupted or non-existent video frames. Ensuring a consistent bus width is vital for the physical data transfer integrity.

Failure to achieve exact synchronization across these three parameters will invariably lead to stream initialization failures, data corruption, or the inability to allocate necessary hardware resources. Developers must treat these configurations as a single, unified setup.

Tegra-capture vi :

```
None
port@0 {
    reg = <0>;
    status = "okay";
    zedx_vi_in0: endpoint {
        status = "okay";
        vc-id = <0>;
        port-index = <0>;
        bus-width = <NUM_LANES>;
        remote-endpoint = <&zedx_csi_out0>;
    };
};
```

Nvcsi :

None

```
channel@0 {
    reg = <0>;
    status = "okay";
    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {
            reg = <0>;
            status = "okay";
            zedx_csi_in0: endpoint@0 {
                status = "okay";
                port-index = <0>;
                bus-width = <NUM_LANES>;
                remote-endpoint = <&zedx_ar0234_out0>;
            };
        };
        port@1 {
            reg = <1>;
            status = "okay";
            zedx_csi_out0: endpoint@1 {
                status = "okay";
                remote-endpoint = <&zedx_vi_in0>;
            };
        };
    };
};
```

Sensor porting :

None

```
zedx_left0: zedx_left_0@28 {

    ...

    AR0234("0", "serial_a", "0")

    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {
            reg = <0>;
            zedx_ar0234_out0: endpoint {
```

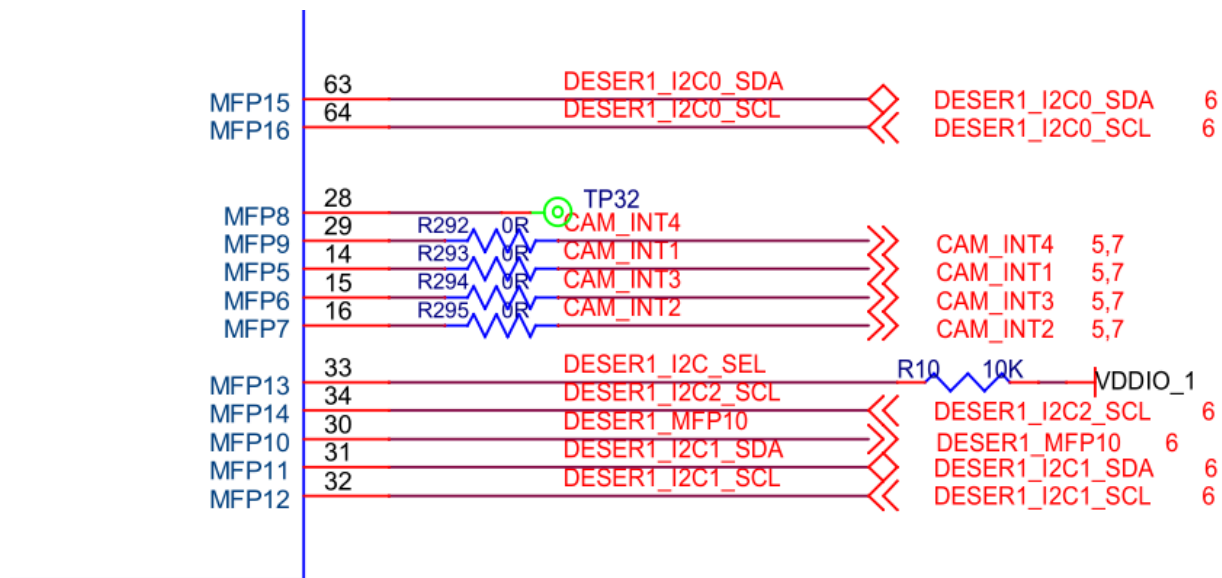
```

        vc-id = <0>;
        port-index = <0>;
        opt-csi-port = "c";
        bus-width = <NUM_LANES>;
        i2c-cc = <0>;
        remote-endpoint = <&zedx_csi_in0>;
    };
};
};
};

```

I2C Control Channels

The MAX96712 has multiple I2C inputs, allowing you to set your cameras on separate channels. This allows you to have up to 2 ZED X One 4K on a single deserializer, for example.



In this configuration, MFP15 and MFP16 utilize I2C0, whereas MFP11 and MFP12 are linked to I2C1. You have the flexibility to determine which I2C bus your cameras connect to by setting the **i2c-cc** variable to either 0 or 1.

Keep in mind that the master control channel (`i2c-cc = 0`) is used to control the deserializer.

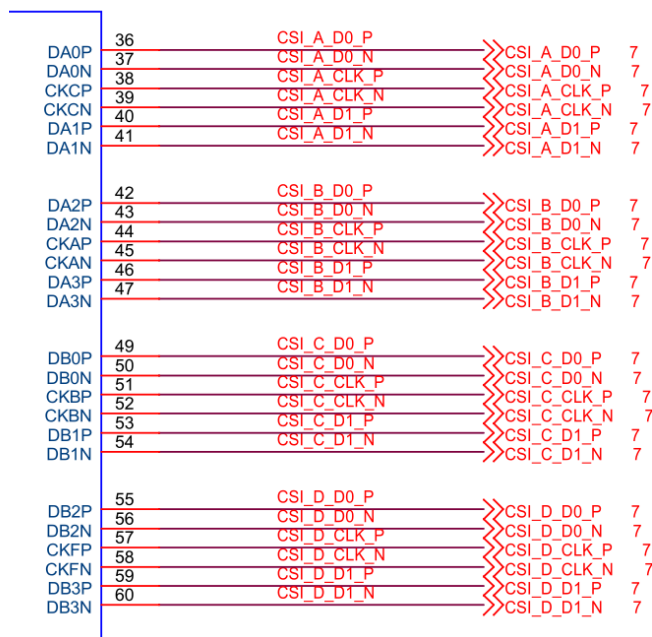
Deserializer MIPI outputs

MAX96712 Deserializer Configuration

The MAX96712 driver currently supports only 2x2-lane outputs, allowing selection of the lane pair for the output stream. The following mapping is used to connect the MAX96712 output ports to the Jetson CSI inputs:

- Port C → CSI_A
- Port D → CSI_B
- Port E → CSI_C
- Port F → CSI_D

Configuration is done in the device tree by setting the `opt-csi-port` value to the desired output port.



MAX9296 Serializer Configuration

For the MAX9296, the output can be configured to use either 2 or 4 lanes on Port A or Port B. This is set using `opt-csi-port = "a"` or `opt-csi-port = "b"`, respectively.

Critical Warning: In the event of modifications, the MIPI CSI porting configuration **must be meticulously applied** across all interconnected kernel entities that manage the video stream pipeline, specifically the `tegra-capture-vi`, the `nvcsi` block, and the `sensor porting` configuration.

